

## TP 3 PL/SQL

### A/ Écrire un bloc PL/SQL permettant de :

1. D'afficher toutes les informations concernant un article donné.

```
SQL> SET SERVEROUTPUT ON;
SQL> connect ghada/ghada
Connected.
SQL> DECLARE
  2     v_noArticle Article.noArticle%TYPE := 1;
  3     v_description Article.description%TYPE;
  4     v_prixUnitaire Article.prixUnitaire%TYPE;
  5     v_quantiteEnStock Article.quantiteEnStock%TYPE;
  6
  7 BEGIN
  8     SELECT description, prixUnitaire, quantiteEnStock
  9     INTO v_description, v_prixUnitaire, v_quantiteEnStock
 10     FROM Article
 11     WHERE noArticle = v_noArticle;
 12
 13     DBMS_OUTPUT.PUT_LINE('Description : ' || v_description);
 14     DBMS_OUTPUT.PUT_LINE('Prix unitaire : ' || v_prixUnitaire);
 15     DBMS_OUTPUT.PUT_LINE('Quantité en stock : ' || v_quantiteEnStock);
 16 END;
 17 /
```

2. D'afficher le nombre total des commandes passées par un client (choisir un numéro qui existe dans votre base).

```
SQL> DECLARE
  2     v_noClient Commande.noClient%TYPE := 1;
  3     v_totalCommandes NUMBER;
  4
  5 BEGIN
  6     SELECT COUNT(*) INTO v_totalCommandes
  7     FROM Commande
  8     WHERE noClient = v_noClient;
  9
 10     DBMS_OUTPUT.PUT_LINE('Total des commandes du client : ' || v_totalC
ommandes);
 11 END;
 12 /
Total des commandes du client : 2

PL/SQL procedure successfully completed.
```

3. Changer le bloc précédent pour pouvoir saisir le numéro du client.

```

SQL> DECLARE
  2     v_noClient Commande.noClient%TYPE;
  3     v_totalCommandes NUMBER;
  4
  5 BEGIN
  6     v_noClient := &v_noClient;
  7
  8     SELECT COUNT(*) INTO v_totalCommandes
  9     FROM Commande
 10     WHERE noClient = v_noClient;
 11
 12     DBMS_OUTPUT.PUT_LINE('Total des commandes du client : ' || v_totalC
ommandes);
 13 END;
 14 /
Enter value for v_noclient: 2
old 6:     v_noClient := &v_noClient;
new 6:     v_noClient := 2;
Total des commandes du client : 1

PL/SQL procedure successfully completed.

```

4. Diminuer le prix unitaire d'un article si la quantité en stock est inférieur à 100.

```

SQL> DECLARE
  2     v_noArticleDim Article.noArticle%TYPE := 1;
  3     v_quantiteEnStockDim Article.quantiteEnStock%TYPE;
  4     v_prixUnitaireDim Article.prixUnitaire%TYPE;
  5
  6 BEGIN
  7     SELECT quantiteEnStock, prixUnitaire
  8     INTO v_quantiteEnStockDim, v_prixUnitaireDim
  9     FROM Article
 10     WHERE noArticle = v_noArticleDim;
 11
 12     IF v_quantiteEnStockDim < 100 THEN
 13         UPDATE Article
 14         SET prixUnitaire = prixUnitaireDim * 0.9
 15         WHERE noArticle = v_noArticleDim;
 16         DBMS_OUTPUT.PUT_LINE('Le prix a été diminué.');
```

5. D'afficher les 5 premiers articles les plus chers.

```
SQL> DECLARE
2     CURSOR c_articles IS
3         SELECT noArticle, description, prixUnitaire
4         FROM Article
5         ORDER BY prixUnitaire DESC
6         FETCH FIRST 5 ROWS ONLY;
7
8     v_noArticleTop Article.noArticle%TYPE;
9     v_descriptionTop Article.description%TYPE;
10    v_prixUnitaireTop Article.prixUnitaire%TYPE;
11
12 BEGIN
13     FOR article IN c_articles LOOP
14         v_noArticleTop := article.noArticle;
15         v_descriptionTop := article.description;
16         v_prixUnitaireTop := article.prixUnitaire;
17
18         DBMS_OUTPUT.PUT_LINE('No Article : ' || v_noArticleTop || ', De
description : ' || v_descriptionTop || ', Prix unitaire : ' || v_prixUnitaireT
op);
19     END LOOP;
20 END;
21 /
```

**B/ Écrire une fonction qui permet de :**

1. Retourner la somme totale d'une commande donnée.

```
SQL> CREATE OR REPLACE FUNCTION getTotalCommandSum(p_noCommand IN Commande.n
oCommande%TYPE)
2     RETURN NUMBER
3 IS
4     v_totalSum NUMBER;
5 BEGIN
6     SELECT SUM(quantité)
7     INTO v_totalSum
8     FROM LigneCommande
9     WHERE #noCommande = p_noCommand;
10
11     RETURN v_totalSum;
12 END;
13 /
```

2. Afficher l'âge d'un client identifié à l'aide de son numéro de téléphone.

```
SQL> CREATE OR REPLACE FUNCTION getClientAge(p_phoneNumber IN Client.noTélép
hone%TYPE)
2 RETURN NUMBER
3 IS
4     v_age NUMBER;
5 BEGIN
6     SELECT TRUNC(MONTHS_BETWEEN(SYSDATE, dateNaiss) / 12)
7     INTO v_age
8     FROM Client
9     WHERE noTéléphone = p_phoneNumber;
10
11     RETURN v_age;
12 END;
13 /
```

C/

Warning: Function created with compilation errors.

Écrire une procédure qui permet de :

1. Afficher le numéro et le nom de tous les clients triés par nom (ordre décroissant).

```
SQL> CREATE OR REPLACE PROCEDURE displayClients
2 IS
3 BEGIN
4     FOR client IN (SELECT noClient, nomClient FROM Client ORDER BY nomC
lient DESC) LOOP
5         DBMS_OUTPUT.PUT_LINE('No Client : ' || client.noClient || ', No
m : ' || client.nomClient);
6     END LOOP;
7 END;
8 /
```

Procedure created.

2. Augmenter la quantité en stock d'un produit donné d'une valeur.

```
SQL> CREATE OR REPLACE PROCEDURE increaseStock(p_noArticle IN Article.noArti
cle%TYPE, p_increaseValue IN NUMBER)
2 IS
3 BEGIN
4     UPDATE Article
5     SET quantitéEnStock = quantitéEnStock + p_increaseValue
6     WHERE noArticle = p_noArticle;
7
8     DBMS_OUTPUT.PUT_LINE('Stock increased for Article ' || p_noArticle)
;
9 END;
10 /
```

Warning: Procedure created with compilation errors.

D/ Écrire un déclencheur qui permet de :

On n'a pas étudié les triggers.